PORTFOLIO · 개인 보고서

입찰메이트 — 검색(Retrieval) 파트

공공기관 제안요청서 질의응답 RAG에서 맡은 일, 마주친 문제, 내린 결정

항목	내용
이름	이재웅
프로젝트	입찰메이트 — 공공기관 제안요청서(RFP) 질의응답 RAG 서비스 · Part3 1팀 중급 프로젝트 (LittleBitAI)
기간	2026. 09. 30 ~ 2026. 10. 21
역할	Project Manager (09.30 ~ 10.02, 초기 베이스라인) → Model Architect (10.06 ~ 10.21, ② 검색 담당)
저장소	github.com/LittleBitAI/rfg_rag · feature/Retrieval 브랜치
근거 자료	팀 최종 발표 자료(38장), 검색 파트 발표 자료(7장), GitHub 커밋 기록, 개인 협업일지(Google Sheets)

1. 프로젝트 개요

입찰 담당자는 평균 8만 5천 자, 그중 절반 이상이 표인 제안요청서 98건에서 '사업기간' 같은 한 줄을 찾아야 합니다. 입찰메이트는 질문을 받으면 관련 문단을 찾아, 그 문단만 근거로 답하고 출처까지 보여 주는 RAG 서비스입니다.

파이프라인은 ① 전처리 → ② 검색 → ③ 검증 → ④ 생성 → ⑤ 평가 다섯 단계로 나뉘며, 저는 ② 검색을 맡아 질문 재작성부터 근거 전달까지의 설계·실험·발표를 담당했습니다.

팀 최종 결과 (Test 943 문항, 설정 고정 후 1 회)

지표	값	의미
정답률	81.2% (Dev 82.5%)	처음 보는 문서의 질문에서도 Dev와 거의 같음
Evidence Coverage	0.926	정답 근거가 답변 모델까지 전달된 비율 — 검색 파트의 최종 성적
Chunk Recall@10	0.934	정답 청크가 검색 상위 10개 안에 든 비율
응답 시간 P50 · P90	3.6초 · 4.9초	질문 하나 전체 기준

지표	값	의미
운영 비용	$0.47 / 1,000건	언어 모델 이용료 (검색은 로컬 GPU)

정답률 = Factual Correctness ≥ 0.5 또는 Key Fact Match = 1 · 출처: 팀 최종 발표 26 · 33장 (평가 결과 보고서)

2. 나의 역할과 기여

영역	한 일	확인할 수 있는 곳
초기 베이스라인 (PM)	End-to-End Naive RAG와 평가 모델 제작, 평가 지표 정의 (Recall@K · MRR · nDCG · RAGAS), HWP 추출 문제를 단일 로더로 해결, 모델 · 청크 크기 · 벡터 DB 선택	협업일지 09.30 ~ 10.02
검색 코드 정비	PDF 명세에 맞춰 Retrieval 수정, rfp.zip(98건) 원본 · 목차 인식 수정, dev 구조에 맞춰 이식, 02 검색 노트북에 기관 라우팅 점검 추가	feature/Retrieval 커밋 11건 (10.13 ~ 10.15)
기관 라우팅	data_list.csv 발주기관 추출 개선 → 98건 모두 기관과 연결, 비교 질문은 기관별로 따로 검색	팀 발표 15 · 16장
하이브리드 검색	KURE + BM25, 점수 가중합 → RRF 전환, 요구사항 번호 일치 조각 우선, 리랭커 실험	검색 발표 3 · 5장
근거 보호 장치	요구사항 · 표 연결, 숫자 · 부정어가 다른 근거 보존, OCR 중복 제거, 색인 일치 점검	검색 발표 6장
실험 · 분석	실험 8 ~ 17 설계와 정리, 로컬 심판 채점 개선, 답변 지표가 낮은 원인 추적, 재작성 추론 강도 비교	팀 실험결과 시트, 검색 발표 7장
발표 · 문서	검색 파트 발표 자료와 대본(팀 최종 발표 검색 장의 출처 자료), 예상 질문 · 답변, README 협업일지 섹션	검색 발표 7장, 팀 발표 12 ~ 17장

3. 진행 과정 (협업일지 기준)

기간	역할	주요 작업	발견한 문제 · 인사이트
09.30 ~ 10.02	PM	Naive RAG 베이스라인, 평가 지표 정의, HWP 추출, 모델 · Dense:BM25 비율 · 청크 크기 · FAISS 선택, 보고서 초안	1회 실행 평균 약 $1.5로 토큰 사용이 많음, HWP 라이브러리 버그 · 한자 처리
10.06 ~ 10.08	MA	오류 자료 조사, BM25 : Dense 비율 실험, CLI 환경 구축, 날짜 · 링크 · 가격 처리, GitHub 베이	골든 데이터셋 필요, 날짜 · 표 · 이미지 처리 과제, 재작성 · 구조 청킹의 API ·

기간	역할	주요 작업	발견한 문제 · 인사이트
		스라인 정비	시간 부담
10.12 ~ 10.14	MA	베이스라인 재설정, 인식 방식을 Qwen → OCR + API로 전환 제안, Retrieval 수정, 토큰 통계 CSV · 체크포인트 설정, 700자 청크 배포, dev · retrieval 브랜치 업로드	OCR 토큰 비용, 구조 기반 청킹 문제, 팀원의 코드 구조 이해 차이
10.15 ~ 10.16	MA	골든 데이터 분석, data_list.csv 개선, 텍스트 겹침 수정, 새 생성 모델로 실험	리랭커 필요, 평가 API 비용
10.19	MA	RFP_OCR 데이터 반영, 리랭커 · Dense 비율 튜닝, RRF 적용	로컬 심판과 API 심판의 점수 차이
10.20 ~ 10.21	MA	실험 결과 정리, 검색 파트 발표 자료 · 대본, 팀 피드백	key_facts · Factual Correctness의 허용 폭이 좁음

4. 검색 파이프라인 설계

질문 하나가 근거가 되기까지 다섯 단계를 거칩니다. 돈이 드는 외부 언어 모델은 1단계뿐이고, 2 ~ 5단계는 로컬 GPU에서 실행합니다.

단계	하는 일	기술 · 설정
① **질문 다듬기**	'그 사업 기간은?'을 앞 대화로 풀어 기관명 · 사업명을 채움. 숫자 · 부정어는 바꾸지 않음	gpt-6-luna, 최근 10턴, reasoning_effort
② **범위 좁히기**	질문 속 발주기관 문서 안에서만 검색 (98건 → 예: 3건)	기관 라우팅 (85개 기관)
③ **두 방식으로 찾기**	뜻이 비슷한 곳 50개 + 단어가 같은 곳 50개	KURE-v1 + FAISS · Kiwi + BM25
④ **합치기**	점수 대신 순위로 합침, 요구사항 번호가 정확히 맞으면 맨 앞	RRF, 1 / (60 + 순위)의 합
⑤ **근거 전달**	2,500토큰 안에서 출처 번호와 함께 전달, 연결 조각 · 중복 정리	표 6조각 이하 연결, 결과당 600토큰

근거를 지키는 여섯 가지 장치

- **나뉜 요구사항 잇기 · 쪼개진 표 잇기:** 번호와 상세, 한 표의 여러 조각을 함께 가져옴
- **'2 초'와 '5 초'는 다른 내용:** 숫자나 '포함 / 불포함'이 다르면 비슷한 문장이어도 둘 다 남김

- **기관 비교는 따로 찾기:** 두 기관 비교 질문은 기관마다 검색해 근거를 공평하게 담음
- **같은 그림은 한 번만 · 색인 일치 점검:** 같은 이미지의 OCR 설명은 하나만, 검색 글이 바뀌었는데 예전 색인이면 멈춤

5. 데이터로 내린 결정

결정	비교한 것	근거 수치	결론
임베딩 모델	Nemotron vs KURE (Dev 329)	Coverage 75.23 vs 73.56% (약 5문항 차), 근거 1개 질문 126 vs 128, GPU 2.64 vs 1.25 GiB, 색인 410 vs 145초	KURE + BM25
결과 합치기	6 : 4 가중합 vs RRF (실험 10 → 12)	context_recall 0.9211 → 0.9251, 근거 속 핵심 사실 0.8987 → 0.9024	RRF (가중치 튜닝 불필요)
기관 필터	필터 끔 vs 켬 (Dev 275)	Chunk Recall@10 0.522 → 0.951, MRR 0.241 → 0.778	필터 사용
재작성 추론	none · low · medium (438문항)	근거 속 핵심 사실 0.896 · 0.893 · 0.893, 재작성 1.77 · 2.16 · 3.11초, $0.068 · $0.139 · $0.179	none
리랭커	켬 vs 끔 (팀 검토, Dev 400)	정답률 83.8%로 같음, P50 5.0 → 6.0초, 오답 65개 중 45개가 답변 단계 실패	최종 설정에서 끔

리랭커는 검색 파트 실험(실험 10 ~ 17)에서 사용했지만, 팀 리랭커 검토에서 정답률 이득이 없어 최종 평가 설정에서는 끄기로 했습니다. 출처: 검색 발표 4 · 7장, 팀 발표 16 · 17 · 35 · 37장, 팀 실험결과 시트

6. 마주친 문제와 해결

각 사례는 문제 → 원인 → 해결 → 결과 순으로 정리했고, 결과는 팀 실험 결과표와 실행 산출물에서 확인한 수치입니다.

6-1. 평가 비용이 실험 횟수를 막았다

문제	RAGAS(LLM 심판)로 484문항을 채점하면 한 번에 $2 ~ 6이 들어, 설정을 바꿔 가며 여러 번 실험하기 어려웠습니다.
원인	검색 · 생성 · 근거성 · 최종 정확도를 모두 유료 LLM이 판정해, 문항이 늘수록 비용이 비례해 커졌습니다.
해결	리랭커 · 임베딩 · 숫자 규칙으로 같은 이름의 지표를 계산하는 로컬 심판을 도입했습니다(API 0원). 심판이 다르면 점수 크기가 달라지므로 '같은 심판끼리만 비교한다'는 규칙을 함께 정했습니다.
결과	심판만 바꾼 실험 9(RAGAS) → 8(로컬): 실행 비용 $2.63 → $0.68 (약 74% 절감). 이후 실험 대부분을 로컬 심판으로 돌릴 수 있었고, 팀 최종 평가도 찾기 · 답 쓰기 채점을 무료 공식으로 바꿨습니다.

6-2. 다른 기관의 비슷한 문서가 검색 결과에 섞였다

문제	제안요청서는 양식이 비슷해, 질문한 기관이 아닌 다른 기관 문서의 조각이 근거로 들어왔습니다.
원인	기관명 기준인 data_list.csv의 발주기관 정보가 고르지 않았고, 통칭과 정식 명칭이 연결되지 않았습니다.
해결	data_list.csv 발주기관 추출을 개선하고, 질문 속 기관명으로 그 기관 문서 안에서만 찾는 라우팅을 적용했습니다. 02 검색 노트북에 라우팅 점검을 추가하고, 두 기관 비교 질문은 기관별로 따로 찾게 했습니다.
결과	98건 모두 기관과 연결(98 / 98). 팀 최종 평가에서 필터 사용 시 Chunk Recall@10 0.522 → 0.951.

6-3. 단위가 다른 두 검색 점수를 합치기 어려웠다

문제	뜻 검색과 단어 검색 점수를 6 : 4로 섞었는데, 질문마다 실제 비중이 흔들리고 가중치를 따로 맞춰야 했습니다.
원인	코사인 유사도(0 ~ 1)와 BM25 점수(상한 없음)는 분포가 달라, 정규화해도 질문마다 결과가 달라졌습니다.
해결	점수 대신 순위만 합치는 RRF로 바꾸고, 요구사항 번호가 정확히 맞는 조각은 항상 맨 앞에 두었습니다. RRF 상수와 리랭커 최소 점수를 설정값으로 빼, API 비용 없이 설정을 비교할 수 있게 했습니다.
결과	실험 10 → 12(합치는 방식만 변경): context_recall 0.9211 → 0.9251, 근거 속 핵심 사실 0.8987 → 0.9024. 성능은 같거나 조금 낮고 가중치 튜닝이 필요 없어졌습니다.

6-4. 문서를 나누면서 근거가 끊기거나 지워졌다

문제	요구사항 번호와 상세, 한 표의 여러 부분이 다른 조각에 들어가 한쪽만 검색되었습니다. '응답 2초'와 '응답 5초' 같은 문장이 중복으로 지워졌고, 같은 그림의 OCR 설명이 여러 번 들어갔습니다.
원인	검색 조각 대부분이 표에서 나왔고, 중복 제거가 문장 유사도만 보고 숫자 · 부정어 차이를 무시했습니다.
해결	같은 요구사항 · 같은 표(6조각 이하)의 연결 조각을 함께 담되 결과당 600토큰으로 제한했습니다. 숫자 · 부정어가 다르면 중복으로 보지 않게 하고, OCR 중복 제거와 색인 일치 점검을 추가했습니다.
결과	조건이 다른 근거가 사라지지 않게 되었고, 근거 속 핵심 사실 비율은 0.90 수준을 유지했습니다.

6-5. 로컬 심판이 맞는 답에 0 점을 줬다

문제	실험 12에서 factual_correctness 0점이 104문항이었는데, 그중 38문항은 정답 핵심 사실이 답에 모두 들어 있었습니다.
원인	정답은 '과업 완료 후 1년 이내'처럼 짧은데 답은 기관명을 붙인 긴 문장이라, 리랭커가 관련도를 낮게 매겼습니다.
해결	답 문장이 정답 전체나 한 줄을 글자 그대로 담으면 뒷받침으로 인정하는 규칙을 추가하고, 숫자 점검은 유지했습니다.
결과	같은 답을 다시 채점한 실험 13 → 14: factual_correctness 0.6016 → 0.6783 (+7.7%p). 같은 문제를 팀 최종 평가도 확인했습니다: 정답 10자 이하에서 핵심 문구 일치 0.833 vs 채점 모델 0.113.

6-6. 답변 지표(0.68)가 낮은 원인을 데이터로 밝혔다

문제	검색 근거에는 정답 핵심 사실이 약 89% 있는데, 최종 답에는 약 65 ~ 68%만 남았습니다.
원인	핵심 사실 878개 추적: 근거에 778개, 답에 574개. 근거에 있었는데 답에서 빠진 210개 중 115개는 표현 · 요약 차이로 글자 대조에서 생긴 감점, 95개는 표 교차 칸 거부와 실제 누락이었습니다.
해결	'절반은 채점 방식, 절반은 답변 단계'로 원인을 나눠 보고했습니다. 답변 프롬프트를 바꾼 실험 13에서 포함률은 0.682 → 0.705로 올랐지만 근거 충실도가 0.839 → 0.473으로 떨어지는 부작용을 확인했습니다.
결과	팀 최종 평가에서도 Test 오답 중 검색 누락 72 < 답변 단계 104로, 병목이 검색보다 답변 단계에 있음이 확인되었습니다.

6-7. 순위 지표가 더 높은 모델을 고르지 않은 이유

문제	팀 비교 보고서에서 다국어 모델 Nemotron이 KURE보다 Recall@10이 4 ~ 5%p 높게 나왔습니다.
원인	비교 조건이 운영 조건과 달랐고, 답변 모델에 실제로 넘어가는 근거 차이는 1.7%p(약 5문항)였습니다.
해결	순위 지표만 보지 않고 실제 전달 근거 · 근거 1개 질문 · GPU 메모리 · 색인 시간을 함께 비교했습니다.
결과	근거 1개 질문은 KURE가 128 vs 126으로 앞서고, Nemotron은 메모리 2배 · 색인 2.8배라 KURE + BM25를 유지했습니다.

6-8. 질문 재작성에 추론을 더 쓰면 좋아질까

문제	질문 재작성에 추론을 쓰고 있었는데, 그만큼 시간과 비용이 들었습니다.
원인	추론 강도가 검색 · 답변 품질에 주는 효과를 측정한 적이 없었습니다.
해결	다른 조건은 고정하고 reasoning_effort만 none · low · medium으로 바꿔 비교했습니다(실험 12 · 16 · 17).
결과	품질은 거의 같았고, 추론을 늘릴수록 질문을 덜 고치고(430 → 394개) 느려지기만 했습니다. 팀 최종 평가는 none으로 진행했습니다. 재작성 단계 시간 1.77초(medium 3.11초), 비용 $0.068(medium $0.179).

6-9. 작은 점수 차이를 믿어도 될까

문제	실험 간 차이가 0.005 안팎일 때 개선인지 우연인지 판단하기 어려웠습니다.
원인	질문 재작성과 답변 생성이 언어 모델이라 같은 설정으로 돌려도 결과가 달라집니다.
해결	같은 설정을 두 번 실행해 비교했습니다. 재작성 질문이 같은 문항은 484개 중 63개뿐이었습니다.
결과	답 속 핵심 사실 평균 차이는 0.001이지만 문항 60여 개가 오르내렸고, '확인할 수 없음' 답도 23 vs 29개였습니다. '답변 지표 차이 0.01 미만은 판단을 보류한다'는 기준을 세워 보고에 적용했습니다.

7. 실험 기록 (팀 실험결과 시트)

실험	바꾼 것	근거 속 사실	답 속 사실	최종 정확도	비용
1	베이스라인 (RFP_v12, mini, RAGAS 심판)	0.855	0.727	0.570	$2.28
8	v12_398 골든 + RFP_OCR, 로컬 심판	0.893	0.664	0.566	$0.68
10	리랭커 적용 + 코드 수정 (6 : 4)	0.899	0.683	0.699	$0.67
12	**RRF 적용**	**0.902**	**0.682**	**0.693**	**$0.67**
13	답변 프롬프트 수정	0.900	0.705	0.602	$0.71
14	13 + 로컬 심판 채점 개선	0.900	0.705	0.678	$0.71
16	**재작성 추론 none**	**0.903**	**0.696**	**0.698**	**$0.59**
17	재작성 추론 medium	0.903	0.681	0.675	$0.71

평가 질문 484문항(Golden 396 + 기존 88) 기준. 실험 1은 RAGAS, 실험 8 이후는 로컬 심판이라 정확도 · 비용을 직접 비교하지 않습니다. '근거 속 사실'과 '답 속 사실'은 글자 대조 지표라 심판과 무관합니다. 팀 최종 평가(Dev 408 · Test 943)와는 질문 세트가 다릅니다.

8. 협업과 커뮤니케이션

- **GitHub:** feature/Retrieval 브랜치에 커밋 11 건(10.13 ~ 10.15) — 전처리 데이터 추가, PDF 명세 기반 Retrieval 수정, dev 구조 이식, 기관 라우팅 점검, 최신 dev · 골든 데이터 반영. 팀 공통 dev 브랜치와 맞춰 가며 작업했습니다.
- **공유 자료:** 700 자 청크 데이터 배포, 토큰 통계 CSV, 노트북 설정 셀로 팀원이 모델 · 비율 · 추론 강도를 직접 바꿔 실험할 수 있게 했습니다.
- **발표:** 검색 파트 발표(7 장)와 대본을 만들었고, 이 자료가 팀 최종 발표 검색 장(12 ~ 17 장)의 출처로 쓰였습니다. 발표 전 예상 질문과 답변도 준비했습니다.

협업에서 겪은 어려움과 대응

어려움 (협업일지)	대응
정기 회의 부족, 팀원 간 에이전트 코드 구조 이해 차이 (10.08 · 10.12)	베이스라인을 다시 세우고 청킹 역할을 재분배, GitHub 환경을 정비
CLI · 코딩 에이전트 사용 미숙 (10.07)	CLI와 에이전트 사용법, 체크포인트 구조를 먼저 익히는 것을 목

어려움 (협업일지)	대응
	표로 잡음
평가 API 비용 부담 (10.16)	로컬 심판 도입, 같은 심판끼리만 비교하는 규칙 합의
PPT 소통 부족 · 글자 간격 문제 (10.21)	팀 피드백 반영, 발표 자료와 대본을 수치 근거와 함께 정리

9. 회고

배운 점

- **숫자는 조건과 함께 본다.** 심판 · 질문 세트 · 실행 횟수가 다르면 같은 이름의 지표라도 비교할 수 없습니다. 리랭커가 검색 실험에서는 도움이 됐지만 최종 정답률은 그대로였던 것도 같은 교훈입니다.
- **한 번에 하나만 바꾼다.** RRF 와 재작성 추론 강도 실험처럼 한 가지만 바꿨을 때 결론이 가장 분명했습니다.
- **지표가 낮으면 원인부터 나눈다.** 답변 지표를 사실 단위로 추적해 채점 방식과 실제 누락을 구분했고, 팀 최종 평가의 실패 분해로도 이어졌습니다.
- **비용도 성능이다.** 로컬 심판, 재작성 none, 리랭커 끄기 결정으로 같은 품질을 더 싸고 빠르게 냈습니다.

아쉬운 점

- 검색 성능이 기관 필터에 크게 기대고 있어(필터 없이 0.522), 질문에 기관명이 없거나 목록에 없는 새 기관 문서에서는 성능을 확인하지 못했습니다.
- 설정마다 1 회씩 실행한 실험이 많아, 작은 차이는 결론을 내리지 못하고 보류했습니다.